

Team Narrative - Different Complementary Specialties

Top 3 Successes:

- There was a natural division of labor from the beginning (**Item.01**)
 - Cause: Each of us has a different technical background and current focus
- Development was quicker and simultaneous across features (**Item.02**)
 - Cause: The project's structure is very modularized (UI, RF model, LLM model)
- There was lots of shared knowledge and explanations between specialties
 - Cause: Updates during weekly meetings and teamwork when features needed to meet (front-end API call to regressor microservice)

Top 3 Pain Points:

- There could be uneven or limited understanding of certain project components (**Item.03**)
 - Cause: Each of us has a different technical background and current focus (e.g. Grant's focus is in ML, so he might understand the predictive models better than Hailey does)
- There could be communication gaps where understanding dipped for one party (**Item.04**)
 - Cause: Misunderstood technical lingo and a strong focus on one's own component
- Integration challenges occurred when features had to connect
 - Cause: Components were built separately from others

What We Learned About Our Teamwork:

- We learned that we function best when every member's roles are clearly defined.
 - For the purpose of this project, that means each member has a focused area in which they specialize. (**Item.01**)
 - This works well when we are able to help each other with our specialties.
 - However, it does not work when someone we are otherwise busy and a given member needs to figure something out of their normal specialization out on their own. (**Item.04**)
- It is helpful to have a strict decision structure.
 - While we rarely have disagreements, we first try to resolve the disagreement among ourselves.
 - If we still cannot come to a decision, assuming it is appropriate, we will ask our project partner for his thoughts.

Evidence Artifacts

Item.01

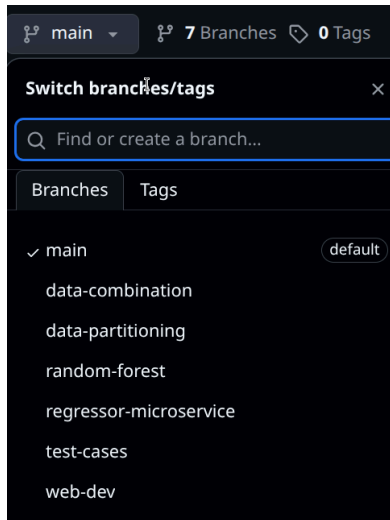


Fig.01 - We have stayed within a natural division of labor since the beginning of last term, as evidenced by our separate branches in our GitHub repository.

Item.02

2. Help Page 'More Questions' Functionality

Status: In-progress

Owner: Hailey

Description: I have updated the 'More Questions' chatbox on the 'Help' page to show questions permanently between systems. I did this by introducing another API call through render called "questions" instead of "single_species". A json file is created, updated, and frequently re-accessed to populate the questions.

Source Link: [2026-02-12-CS.032-Sprint8-ProgressReport.docx](#)

Supporting Artifact: Item.03

3. Decision Tree Regression

Status: Completed

Owner: Grant

Description: The final version of the Decision Tree Regression has been completed. The model size seems to adequately match the size of the dataset. Future steps will involve integrating the model into a microservice such that it can make predictions upon requests from the frontend.

Source Link: [2026-01-13-CS.032-Sprint6-ProgressReport.docx](#)

Supporting Artifacts: Item.04

Notes: For information about the XGBoost library, check out the manpages here: [XGBoost](#)

Fig.02 - Maintaining a division of labor according to specialty allowed us to get work done quicker and allowed different specialities to make significant progress within the same sprint, as evidenced by this sprint work log in which the web development and regressor model saw good progress.

Item.03

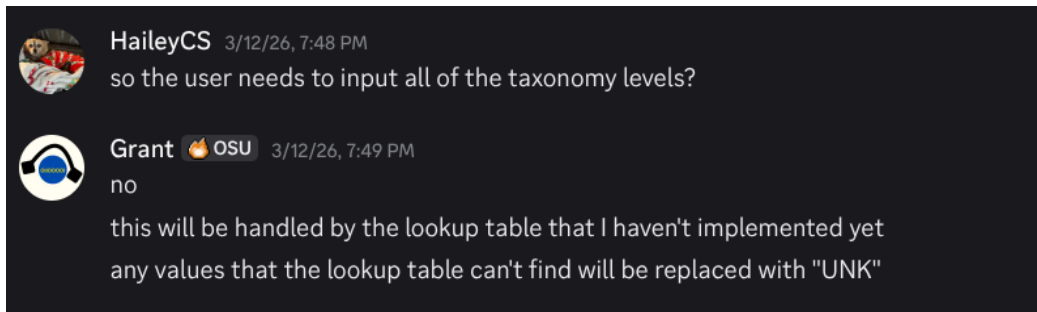


Fig.03 - Hailey was not knowledgeable on the microservice components Grant was implementing, so when she had to edit the javascript to work with them, there was a lack of understanding on how it worked.

Item.04

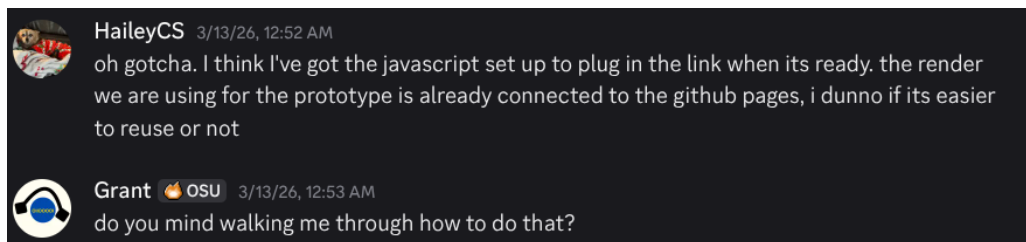


Fig.04 - Because Hailey wrote the initial API request and deployed it on Render, there was no cross-specialty understanding so Grant could do the same thing later on when it was necessary.

Team Action Plan

Action.01 - Implement Confidence Interval Returns from Regressor Model

Owner: Grant

Timeframe: Complete by Sprint 12 (end of week 4 of spring term).

Recognition Cues: Confidence interval is viewable in the UI upon return of mass prediction.

Action.02 - Complete the LLM-Based Prediction Model

Owner: Anish

Timeframe: Complete by sprint 11

Recognition Cues: Output is generated along with the Regressor model in the UI.

Action.03 - Integrate All “Last-Touch” UI Features

Owner: Hailey

Timeframe: Starting next sprint

Recognition Cues: Everything on [this document](#) has been implemented; our project partner is satisfied with every UI functionality; interviews are consistently conducted with little-to-no constructive feedback.

Individual Pages

Author: Hailey

Wins

Win.01 - Increased the usability of the UI for people of varying digital literacy levels

I introduced features to the UI that allow for anyone to effectively interact with the website. Got feedback from people of differing digital backgrounds via interviews to inform my choices.

Examples:

- Implemented a pop-up tool-tip tutorial system that users may choose to opt into that describes how to use all home page features
- After interviewing users of lower digital literacy and getting their feedback, changed the 'x' close button on the intro modal to a more intuitive 'Get Started' button

Win.02 - Modularized html, css, and js files for easier updates

Started moving some code for different aspects of the UI into their own individual files to allow for easier GitHub merges, code edits, and a smoother transition to any future team who may work on this project

Examples:

- The tutorial functionality has its own javascript file that controls its movements and its own css file to determine the tool-tip styling
- Each tab in the navigation bar has its own html file and a separate css file

Gaps

Gap.01 - CSS is written inconsistently

I have not been consistent with the units in which I write the CSS (% , px, etc.); because of this, the UI may look perfectly fine on my computer, but poor on a different one.

Example:

- During the course of the second-phase interviewing, I pulled up our website on a computer that was not mine. This computer had a bigger screen than mine does, so the modals and navigation bar looked very small; this made the whitespace seem all-encompassing and the UI overall look not good.

Improvement Commitments

IC.01 - Perform consistent PRs from the web-dev branch that are smaller in scale

Plan: Break UI updates into few-feature PRs instead of large, multi-component merges.

Check-in: At least one small PR per sprint, reviewed at weekly standup meetings to ensure web-dev content is seen in the main branch. This was not followed this past term, so I want to stay committed to this during spring term.

IC.02 - Make CSS styling more consistent and scaleable (directly related to known gap)

Plan: Comb through CSS code and standardize a specific unit; also, check how the UI looks on different screen sizes while updating.

Check-in: At the end of each sprint, ensure that that sprint's updates look good on a variety of screen sizes; if possible, when conducting any future interviews, open the UI on a computer that isn't my own. Present to the team at the next standup meeting to ensure accountability.

Risk and Mitigation

Risk: Modularizing the UI into many separate files may introduce integration issues.

Description: Because I have started to split the HTML, CSS, and JS code into modular components, it is easier to manage the code, but it also has increased the chance of missing imports or breaking dependencies between files.

Trigger: Navigation, tutorial features, or other UI components fail to load or function correctly after pushes or merges.

Mitigation Action: Perform smaller, more frequent pull requests and test each component after integration into the main branch to ensure all dependencies remain properly connected.

Success Cue: UI components load and function correctly after merges, and no functionality breaks when new files are added.

Author: Grant

Wins

Win.01 - Completed first version of both main responsibilities

Examples:

- Completed XGBoost regressor training script
- Completed first functional version of the look up table microservice
- Completed first functional version of the regressor microservice

Gaps

Gap.01 - I need to clarify specific system architecture with respect to how we are handling multiple simultaneous requests.

Examples:

- The original plan was to use a multi-threaded queue to handle simultaneous prediction requests.
- Currently, the architecture circumvents the multi-threaded queue.
- My implementation and planning in this area have been inconsistent.

Improvement Commitments

IC.01 - More frequent pull requests.

Plan: For any feature update, there should be an associated pull request. Generally, each PR should be only 1 major change rather than many smaller updates.

Check-in: Each sprint should contain at least one pull request. I expect this to indicate that PRs are more specified to individual features rather than multiple large changes.

IC.02 - Better structured explanations for collaborative aspects of code.

Plan: I will do a better job of formalizing my expectations for collaborative parts of the system architecture (APIs, shared memory, etc.).

Check-in: Each feature explanation should be formatted as follows: Title, Reasoning, Shared Communication Channels, Implementation Suggestions. This should help clarify any inconsistencies in understanding between various system interfaces.

Blocker Prevention

Risk: We fall behind in completing the most important project requirements.

Mitigation Step: Add a final checklist for all the remaining tasks which **MUST** be completed before the end of our work period.

Trigger: If there are any remaining tasks by Sprint 13 (week 6 of spring term), we should heed the importance of this list.

“Helped” Cue: We will know that this worked if we are able to complete each item on the checklist before Sprint 13, so we can begin focusing on auxiliary requirements.

Author: Anish

Wins

Win.01 - Tested the xgboost regressor with partial data.

Checked if the XGboost regressor was giving outputs with clear phylogenetic and body mass data.

Win.02 - Created API framework

I created the body framework to call our API. This will later show the expected mass output along with the XGBoost regressor model.

Gaps

Gap.01 - Need to find more efficient model which will give us more realistic mass outputs.

Search for other AI models which will give use a better mass output while using less tokens.

Improvement Commitments

IC.01 - Perform smaller data tests to improve time efficiency

Plan: Break data into smaller parts and run them parallelly using Dask dataframe

Check-in: At least a 10th of the data being merged in each of the samples per 5min.

IC.02 - Focus more of effort on the LLM predictive component.

Plan: Work on finding existing LLM models which work with biological data, as well as look at token efficient models in order to reduce budget cost.

Check-in: There should be a functional version of the LLM predictive model, which achieves at least 80% accuracy over the test dataset by the end of the second sprint of winter term.